(파이토치 트랜스포머를 활용한) 자연어 처리와 컴퓨터 비전 심층학습

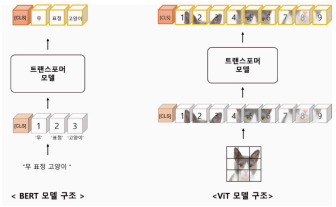
▼ 10장

VIT(Vision Transformer)는 이미지를 자연어 처리 방식럼 분류해 보려는 시도에 의해 만들어짐. 즉 이미지 분류의 task이긴 하지만 CNN 모델의 합성곱 계층 방법을 사용하지 않고 트랜스포머 모델에서 사용되는 셀프 어텐션을 적용해 전체 이미지를 한 번에 처리한다. 하지만 ViT 모델은 이미지 패치가 순차적으로 입력되어(sequential하게) 구조의 이미지 특성을 온전히 반영한다고 보기 어렵기 때문에 이를 해결하기 위해 물체의 크기나 해상도를 계층적으로 학습하는 스윈 트랜스포머(Swin Transformer)와 CvT(COnvolutional Vision Transfoermr) 모델이 제안됐다.

- 스윈 트랜스포머는 local window를 활용해 각 계층의 어텐션이 되는 패치의 크기와 개수를 다양하게 이미지의 특징을 학습한다.viT 구조와 비교할 때 기존 셀프 어텐션을 local window 안에 어텐션, 로컬 윈도 간의 어텐션으로 수행하여 이미지의 특징을 계층적으로 학습시킨다. 이 어텐션 함수에는 상대적 위치 편향을 반영하여 어텐션 값 자체에 위치적 정보를 포함시킨다.
- CvT는 기존 합성곱 연산 과정 ViT에 적용한 모델로, 저수준 특징과 고수준 특징을 계층적으로 반영 할 수 있다. 또한 어텐션 연산 과정에서 쿼리(Query, Q), 키(Key, K), 값(Value, V) 중 키와 값을 기존 특징 벡터보다 축소해 계산 복잡도를 감소시켰다.

ViT :

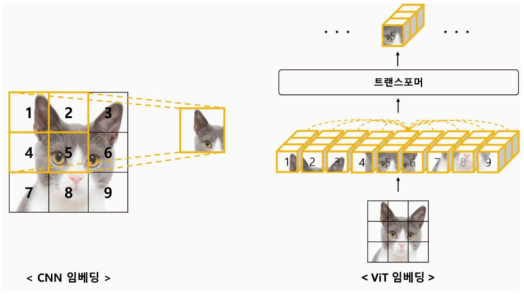
이전 컴퓨터비전에서 합성곱 신경망에 트랜스포머 모델의 셀프 어텐션 모듈을 적용한 모델이 있었는데, ViT는 트랜스포머 구조 자체를 컴퓨터비전 분야에 적용한 것이다.



BERT는 문장보다 작은 단위의 토큰들이 순차적으로 입력됨 ViT 모델은 이미지가 격자로 작은 단위의 이미지 패치로 나뉘어 순차적으로 입력된다는 차이가 있음.

- 합성곱 신경망 vs ViT모델 비교 : 두 모델 모두 이미지의 특징을 잘 표현하는 임베딩을 만들고자 하지만 과정에서 차이가 존재한다.

합성곱 신경망	ViT 모델
이미지 패치 중 일부만 선택하여 학습하여 이미지의 전체 특징을 추출	이미지를 작은 패치들로 나눠 각 패치 간의 상관관계를 학습하여 이미지의 전체 특징 추출
	모든 이미지 패치가 학습에 관여하여 높은 수준의 이미지 표현 제공
좁은 receptive field를 가졌기 때문에 전체 이미지 정보를 표현하는 데 수많은 계층이 필요	어텐션 거리(쿼리 벡ㅌ터와 값 벡터 사이의 유사도를 내적으로 계산)를 계산하는데 오직 한 개의 ViT 레이어
픽셀 단위로 처리	패치 단위로 처리하여 더 작은 모델로도 높은 성능)
이미지의 공간적인 위치 정보를 고려함	패치 간의 상대적인 위치 정보만을 고려하여 이미지 변환에 취약



예를 들어 고양이의 눈 특징을 학습하기 위해서는 CNN은 1,2,3,4만 사용되지만 ViT모델은 모든 이미지 패치가 학습에 관여한 것이다.

- ViT의 귀납적 편향 : 귀납적 편향이란 학습 알고리즘이 새로운 데이터에 대해서도 올바른 출력을 에측하기 위해 내리는 기본적인 가정이나 선호도를 뜻함. (즉 무작위로 모든 가능성을 탐색하는 대신 특정 형태의 패턴을 먼저 학습) 예를 들어 이미지 데이터의 경우 공간적인 정보를 학습하기 때문에 지역적 편향이 존재하며, 시계열 데이터의 경우 순차적 편향이 존재한다. VIT 모델의 경우 다양한 쿼리, 키, 값의 임베딩 형태로 일바놔된 관계를 학습하기 때문에 귀납적 편향이 거의 없고, 대용량 데이터에 대해서 이미지의 특징드르이 관계를 잘 학습하는 모델로 알려져 있다.
- 구조 : 패치 임베딩 + 인코더
  - 패치 임베딩 : 입력 이미지를 트랜스포머 구조에 맞게 일정한 크기의 패치로 나눈 다음 각 패치를 벡터 형태로 변환하는 과정
    - 작은 패치로 분할하기 위해 이미지의 크기를 맞추는 전처리를 수행하고, 전체 이미지를 패치 크기로 분할해 시퀄셜 배열을 만듦.
    - 커널 크기와 sride를 설정해야 한다. (if 이미지 크기 9x9, 커널 크기 3x3, 간격3 이라면 총 9개의 이미지 패치가 생성됨) patch size = (image size-kernel sie)/stride, 패치 개수 = (이미지 크기-커널크기+간격)/패치크기^2
    - 분류 토큰 : 이미지 패치들을 나열할 때 배열 가장 왼쪽에 분류 토큰(Special Classification Tooken, CSL)을 추가함. CLS는 전체 이미지를 대표하는 벡터로 특정 문제를 예측하는 데 사용됨.
    - 위치 임베딩 : 패치의 위치 정보를 임베딩 벡터로 변환하고 기존 이미지 패치 벡터들과 더해 인접한 패치 간의 관계를 학습
    - 계층 정규화
  - 인코더 : 각 패치와의 관계 학습

- N개의 인코더 레이어를 반복적으로 적용한 후, 마지막 레이어에서는 특징 벡터인 CLS를 추출한다. 예를 들어 고양이인지를 구분하는 모델을 학습한다면 ViT모델은 시퀀셜 산출물을 모두 사용하는 것이 아니라 CLS만 사용해서 분류 문제를 풀어내는 것. 따라서 CLS를 순방향 신경망(또는 완전 연결 계층)에 연결하고 분류하는 방법으로 모델이 학습됨.

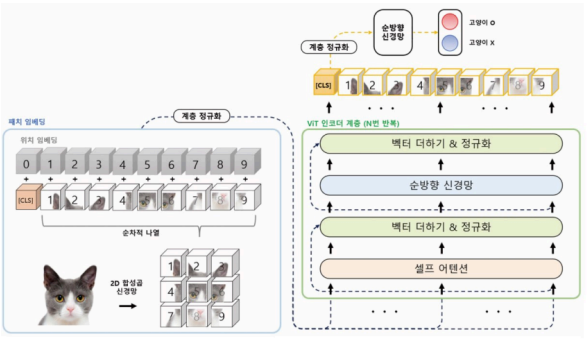


그림 10.4 ViT 모델 구조

- TrainingArguments

표 10.2 학습 매개변수 클래스(TrainingArguments)

매개변수	의미
output_dir	체크포인트 저장 경로
save_strategy	체크포인트 저장 간격 설정 ▪ no: 저장 안 함 ▪ steps: 스텝마다 ▪ epoch: 에폭마다
save_steps	save_strategy= "steps"일 때 스텝 간격 설정
save_total_limit	체크포인트 최대 저장 개수
evaluation_strategy	체크포인트 평가 간격 설정 ▪ no: 저장 안 함 ▪ steps: 스텝마다 ▪ epoch: 에폭마다

매개변수	의미
learning_rate	초기 학습률
per_device_train_batch_size	학습 배치 크기
per_device_eval_batch_size	평가 배치 크기
num_train_epochs	학습 반복 수
weight_decay	가중치 감쇠
load_best_model_at_end	모델 불러오기 시 최상의 모델 선택 여부
metric_for_best_model	최상의 모델 선정 기준이 되는 평가 방식 설정 ▪ accuracy: 정확도 ▪ precision: 정밀도 ▪ f1: F1 점수 ▪ mae: 평균 절대 오차 ▪ mse: 평균 제곱 오차 ▪ rmse: 평균 제곱근 편차
logging_dir	로그 저장 폴더
logging_steps	로그 출력 간격
seed	학습 시작 시 설정되는 무작위 시드
fp16[5]	Mixed precision 사용 여부

- 코드는 그냥 나중에 시간 나면 복붙

```
```

Swin Transformer

: 기존 트랜스포머 모델은 고정된 패치 크기로 인해 세밀한 예측을 필요로 하는 임론적 분할과 같은 작업에 적합하지 않다는 것과 셀프 어텐션은 입력 이미지 크기에 대한 2차 계산 복잡도를 가져 고해상도 이미지를 처리하기가 어렵다는 문제가 있다. 이러한 문제를 극복하기 위해 계층적 특징 맵을 구성해 1차 계산 복잡도를 갖는 스윈 트랜스포머가 등장했다. **스윈 트랜스포머**는 shifted window라는 기술을 이용해 입력 이미지를 일정한 크기의 패치로 분할하고 이 패치들을 쌓아 윈도우를 구성한다. 구성된 윈도 영역만 셀프 어텐션을 계산한다. 이 방식은 기존 트랜스포머 모델과는 다르게 패치를 겹쳐 더 큰 효율성을 제공하고 계층마다 어텐션이 수행되는 패치의 크기와 개수를 계층적으로 적용해 처리하므로 높은 해상도와 다양한 크기를 가진 객체를 효율적으로 처리할 수 있다. 이러한 특징을 바탕으로 객체 탐지나 객체 분할 등과 같은 작업에서 백본 모델로 사용된다.

- ViT와 스윈 트랜스포머 차이 : ViT 모델은 획일적이 패치 크기와 위치에 대한 제약 때문에 다양한 이미지 크기와 종횡비를 가진 데이터 세트에 대해서는 적용이 어렵다. 또한 패치 단위로 입력을 받기 때문에 이미지의 공간 정보가 완전히 보존되지 않을 수 있다. 이미지의 패치 수가 증가하면 학습 데이터의 크기도 커지기 때문에 모델 학습에 필요한 계산량이 증가한다. 이로 인해 대규모 데이터세트에서는 학습에 필요한 컴퓨팅 자원을 많이 사용하게 되는 단점이 있음.

  이러한 한계점을 보완하기 위해 스윈 트랜스포머는 Local Window를 활용해 물체의 크기나 ㅐ상도를 계층적으로 학습한다. 로컬 윈도는 스윈 트랜스포머에서 입력 이미지를 처리하기 위해 사용되는 고정 크기의 작은 윈도우로 시프트 윈도 기술으로 입력 이미지를 일정한 크기의 패치로 분할고, 특징 맵을 이동시켜 다음 계층에 사용함으로써 패치의 위치를 더 유연하게 다룬다. 이를 통해 스윈 트랜스포머는 ViT보다 더 높은 정확도와 더 적은 매개변수 수를 가지면서도 더 넓은 이미지 크기와 종횡비를 다룰 수 있다.

  이러한 계층적 접근 방식은 로컬 윈도가 각 패치의 세부 정보에 집중하게 되고, 시프트 윈도를 통해 전역 특징을 확인할 수 있으므로 고해상도 이미지를 효율적으로 처리할 수 있다.

표 10.4 ViT와 스윈 트랜스포머 비교

	ViT	스윈 트랜스포머
분류 헤드	[CLS] 토큰 사용	각 토큰의 평균값 사용
로컬 윈도 사용	X	O
상대적 위치 편향 사용	X	O
계층별 패치 크기	동일함	다양함

- 모델 구조 : 이미지를 **패치 파티션**으로 구분할 후, **선형 임베딩, 스윈 트랜스포머 블록**, **패치 병합** 연산이 반복적으로 수행된다. 먼저 입력 이미지를 일정한 크기의 패치로 분할하고, 각 패치에 대해 선형 임베딩을 수행한다. 이를 통해 각 패치는 고정된 차원의 벡터로 변환된다.

  다음으로, 분할된 패치들을 기반으로 스윈 트랜스포머 블록을 구성한다. 이 블록은 어텐션 계층, 계층 정규화, 다층 퍼셉트론 등을 포함하여 패치 간 상호작용을 수행한다.

  마지막으로 패치를 병합해 전체 이미지에 대한 분류를 수행한다. 분할된 패치들을 순차적으로 합치면서 이미지의 전체적인 정보를 추출하는 방식을 사용한다.

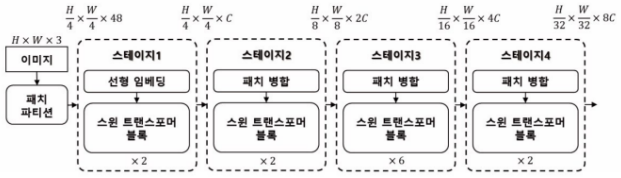


그림 10.7 스윈 트랜스포머 구조

스윈 트랜스포머 모델은 일반적으로 네 개의 스테이지로 구성돼 있으며, 각 스테이지는 서로 다른 해상도와 패치 크기를 갖는다. 스테이지1에서는 패치들이 선형 임베딩된 후 스윈 트랜스포머 블록에 전달되며, 스테이지234에서는 패치 병합후 스윈 트랜스포머 블록에 전달된다. 패치 병합은 모델 매개변수를 줄이기 위해 이미지 텐서[C, H, W]를 [2C, H/2, W/2]로 재정렬해 저차원 임베딩을 수행한다. 그리고 스테이지 1, 2, 3, 4에서 공통적으로 사용되는 스윈 트랜스포머 블록은 우니도 격자 안에 있는 패치 내부의 관계, 윈도 격자 간의 관계를 학습하는 두 가지 어텐션 모듈로 이뤄져 있다.

- 패치 파티션 : 입력 이미지를 작은 사격형 패치로 분할해 처리하는 방식이다. 분할된 패치는 트랜스포머 계층의 입력으로 사용되며, 입력 이미지의 공간 정보를 보존하고 트랜스포머 계층에 대한 계산 효율성을 높인다. 일반적인 이미지 텐서 [C, H, W]의 구조에서 패치 파티션을 통해 로컬 윈도를 추가한다.

  그림 10.8의 M은 로컬 윈도의 크기를 의미한다. 이미지 텐서의 차원이 [3, 32, 32]이라면 패치의 가로(P)와 세로(P)는 4가 된다.

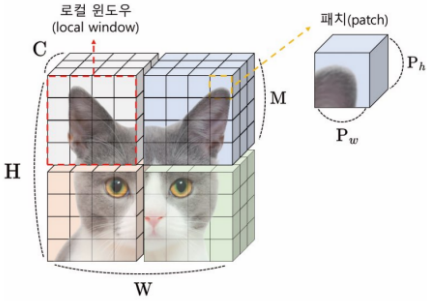


그림 10.8 패치 파티션 구조

- 패치 병합 : 인접한 패치들의 정보를 저차운으로 축소한다. 모델 매개변수의 수를 줄이기 위해 이미지 패치 텐서 [C, H, W]를 [2C, H/2, W/2]로 재정렬한 후, 2C의 차원을 저차원으로 임베딩한다. 예를 들어 C가 3개, 이미지 패치(HxW) 64개로 구성된 텐서 [3, 8, 8]을 가정해봤을 때 정의된 [2C, H/2, W/2]에 따라 [6, 4, 6]로 재정렬되며, 증가된 6차원 채널을 원래 3차원으로 임베딩해 산출물[3, 4, 4] 텐서를 생성한다.
- 스윈 트랜스포머 블록 : 패치 파티션 이후 네 개의 스테이지 안에서 반복 적용되며, 선형 임베딩 또는 패치 병합 이후에 수행된다. 계층 정규화, W-MSA(window multi-head self attention), MLP, SW-MSA(Shifted Window Multi-head Self-Attention) 등으로 구성된다.

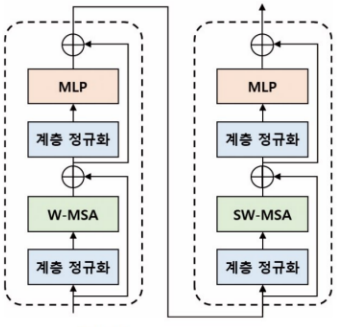


그림 10.9 스윈 트랜스포머 블록 구조

스윈 트랜스포머 블록 안에서는 트랜스포머 모델에서 사용되던 MSA이 아닌 W-MSA와 SW-MSA로 구성되어 순차적으로 수행한다.

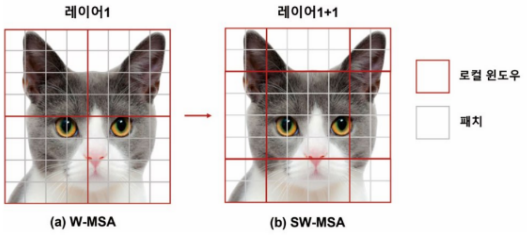


그림 10.10 W-MSA와 SW-MSA 어텐션 영역 비교

- W-MSA(**로컬 윈도우 내 패치 간의 셀프 어텐션**)란 로컬 윈도를 사용하는 멀티 헤드 셀프 어텐션으로, 입력 특징 맵을 중첩되지 않게 나눈 다음 각 윈도에서 독립적으로 셀프 어텐션을 수행한다. 이를 통해 이미지 내의 전체적인 어텐션을 수행하고, 특정 위치에 대한 정보를 추출한다. 또한 각 윈도 내 지역적인 특징을 분석할 수 있으며, 전체 입력에 대한 셀프 어텐션 비용을 2차 계산 복잡도에서 1차 계산 복잡도로 줄인다. 다만 윈도 내부의 이미지 패치 영역에만 셀프 어텐션을 수행한다는 단점이 존재한다. 이를 보완하기 위해 나온 게 SW_MSA이다.

  계산량이 많아보일 수는 있으나 효율적인 배치 계산(Efficient Batch Computation)을 통해 연상량을 감소시켰다. 로컬 윈도에서 연산을 수행하는 과정에서도 배치 축을 사용할 수 있어서 연산 속도가 빨라진다.

- SW-MSA(**로컬 윈도우 간의 셀프 어텐션**)는 윈도 사이의 연결성 구축해 윈도 간의 관계성 정보를 추출한다. 윈도를 가로 또는 세로 방향으로 이동한 후, 인접한 윈도 간의 셀프 어텐션을 계산한다.

  SW-MSA는 W-SMA에서 사용된 로컬 윈도 개수가 더 많아 더 많은 셀프 어텐션 연산을 요구한다. 이 문제를 해결하기 윟 순환 시프트(CYclic Shift)와 어텐션 마스크(Attention Mask)를 사용했다. 순환 시프트는 로컬 윈도가 이동할 때 이동한 위치의 정보를 이전 위치에서 가져오는 것을 말하며, 로컬 윈도 사이즈 M보다 작은 M/2 만큼 로컬 윈도들을 이동시킨다. 이때 이동된 위치에서 연산을 수행하지 않도록 어텐션 마스크를 사용해 방지한다. 마지막으로 순환 시프트 단계에서 셀프 어텐션을 수행한 다음 역순환 시프트(Reverse Cyclic Shift)를 사용해 원래 로컬 윈도 구조로 복원한다. 이러한 방식을 통해 SW-MSA 방식에서 로컬 윈도 간의 셀프 어텐션 연산을 효율적으로 수행한다.

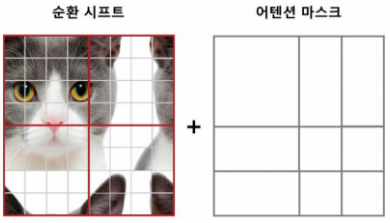


그림 10.12 순환 시프트 단계에서 마스크를 활용한 셀프 어텐션 구조

  위의 그림과 같이 어텐션 마스크를 사용하면 총 9개의 윈도 파티션에 대해 9번의 셀프 어텐션 값을 구해야했지만, 4개의 윈도 파티션에 대한 3개의 셀프 어텐션 값만으로 모든 로컬 윈도 간의 셀프 어텐션 값을 계산할 수 있다. 이는 어텐션 마스크가 순환 시프트를 통해 이동된 윈도에 대한 정보를 보존하면서도 불필요한 연산을 줄이고 효율ㅈ거인 계산을 가능한다는 장범을 가진다.

- 기존 ViT와 다른 점은 스윈 트랜스포머ㅣ에서 사용한 셀프 어텐션을 상대적 위치 편향(Relative Position Bias)를 고려한다는 것이다. 상대적 위치 편향은 로컬 윈도 안에 있는 패치 간의 상대적인 거리를 임베딩하는 목적을 가진다.
- 상대적 위치 편향 : 이미지 패치들이 1부터 4까지 주어진다면 패치 간의 상대적 거리는 X축과 Y축 두 가지 방식으로 표기할 수 있다. X축 방식은 두 패치 간의 가로 방향 거리를 의미하며, Y축 방식은 세로 방향걸리를 의미한다.

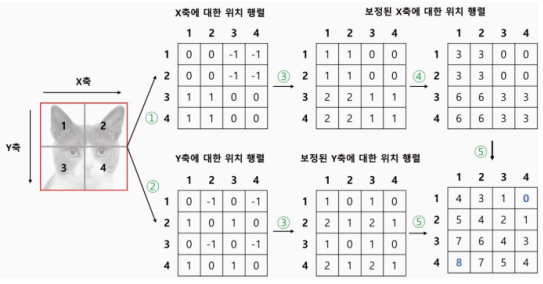


그림 10.13 상대적 위치 편향 계산 과정

이미지 패치들이 1부터 4까지 주어진다면 패치 간의 상대적인 거리는 X축과 Y축 두 가지 방식으로 표기할 수 있다. X축 방식은 두 패치 간의 가로 방향 거리를 의미하며, Y축 방식은 세로 방향 거리를 의미한다.

그림 10.13의 X축에 대한 위치 행렬은 같은 X축에 놓여 있으면 상대적 거리가 0이 되며, 아래로 떨어져 있다면 1, 위로 떨어져 있으면 −1이 된다. 마찬가지로 Y축에 대한 위치 행렬은 같은 Y축에 놓여 있으면 상대적 거리가 0이 되며, 오른쪽으로 떨어져 있으면 −1, 왼쪽으로 떨어져 있으면 1이 된다.

- 스윈트랜스포머 블록은 계층 정규화 → W-MSA(SW-MSA) → 계층 정규화 → MLP의 순서로 수행된다.
  - layernorm_before은 첫 번째 계층 정규화, attention은 W-MSA(SW-MSA), layernorm_after는 두 번째 정규화, intermediate와 output은 MLP를 의미한다
  - MLP 계층을 살펴보면 두 개의 연속된 선형 변환 계층과 GELU(Gaussian Error Linear Unit) 활성화 함수로 구성된다. GELU는 ReLU와 유사한 형태를 가지며, 가우시안 분포에 대한 누적 분포 함수이다. GELU는 입력 값이 0 주변에서 부드럽게 변화한다.ㅌ

```
SwinLayer(
  (attention): SwinAttention(
    (q_proj): Linear(in_features=96, out_features=96, bias=True)
    (k_proj): Linear(in_features=96, out_features=96, bias=True)
    (v_proj): Linear(in_features=96, out_features=96, bias=True)
    (o_proj): Linear(in_features=96, out_features=96, bias=True)
    (relative_position_bias): SwinRelativePositionBias()
  )
  (layernorm_before): LayerNorm((96,), eps=1e-05, elementwise_affine=True)
  (layernorm_after): LayerNorm((96,), eps=1e-05, elementwise_affine=True)
  (mlp): SwinMLP(
    (activation_fn): GELUActivation()
    (fc1): Linear(in_features=96, out_features=384, bias=True)
    (fc2): Linear(in_features=384, out_features=96, bias=True)
  )
  (dropout): Dropout(p=0.0, inplace=False)
  (drop_path): Identity()
)
```

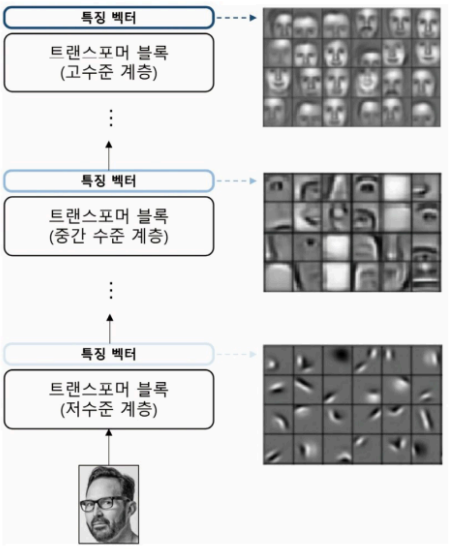
- 패치 병합은 입력 패치를 절반으로 분할하는 reduction과 계층 정규화를 수행한다. 패치 병합 과정을 통해 3,136개의 패치가 784개로 분할한다. 분할된 패티는 네 개로 나뉘어 (N, 28, 28, 96)의 텐서가 생성된다. 이 네 개의 텐서를 채널에 병하바면 96*4=384가 된다. 이렇게 하면 채널이 줄어들지 않으므로 선형 변환 계층을 통해 채널의 수를 절반(384→192)으로 조정한다. 그러므로 patch_merge 결과 차원은 [N, 784, 192]가 된다. 이와 같은 방식으로 총 네 개의 스테이지를 진행한다. 스테이지가 지날수록 패치 크기와 수는 많아지며 토큰의 차원은 두 배씩 증가한다. 마지막 스테이지를 통과하면 [N, 49(7×7), 768]이 된다. 이 특징맵이 풀링 계층을 통과해 클래스를 예측하게 된다.

CvT((Convolutional Vision Transformer)

합성곱 신경망 구조에서 활용하는 계층형 구조를 ViT에 적용한 모델이다.

ViT의 셀프 어텐션은 모든 패치에 대해 동일한 크기와 간격을 사용하므로 이미지의 물체 크기나 해상도를 계층적으로 학습하기가 어렵다. 스윈 트랜스포머가 트랜스포머 블록마다 다른 윈도 크기를 사용해 이미지의 계층적 특징을 학습했다면, CvT는 합성곱 신경망에서 사용하는 계층적 구조를 ViT에 적용해 이미지의 local feature와 global feature를 모두 활용해 비교적 적은 수의 매개 변수로 높은 성능을 보인다.

이런 CvT 구조는 합성곱 계층의 지역 특징을 추출하고 ViT의 셀프 어텐션 계층으로 전역 특징을 결합해 이미지를 처리한다. 그러므로 CvT는 스윈 트랜스포머보다 더 적은 수의 매개변수로도 비슷한 수준의 성능을 보인다. CvT 모델은 토큰에 대한 위치 임베디을 적용하지 않아도 모델의 성능 저하 없이 다양한 해상도를 가진 이미지를 처리할 수 있다.



얼굴 이미지가 CvT 모델의 입ㄹ1으로 사용된가면 저수준 계층에서 선 또는 점에 대한 특징을 표현하고, 중간 수준에서는 눈이나 귀와 같은 특징을 표현하며, 고수준에서는 얼굴 구조에 대한 특징을 표현한다. 그러므로 CvT의 계층형 구조는 물체의 크기나 해상도를 더 잘 표현한다고 할 수 있다.

- 합성곱 토큰 임베딩(Convolutional TOken Embedding) : 2D로 재구성된 토큰 맵에서 중첩 합성곱 연산을 수핸하는 임베딩ㅇ을 의미한다. 이미지 패치를 2D 합성곱 임베딩을 사용해 쿼리, 키, 값 임베딩으로 변환한다. 이렇게 변환ㄴ된 임베딩을 사용해 셀프 어텐션을 계산하고 출력을 생성한다.

  그림과 같이 이미지에 3x3 커널 크기, 1 간격의 커널을 적용하면 패치 크기ㅡㄴ 3x3으로 생성된다. 각각의 패치를 쿼리, 키, 값에 대해 합성곱 연산을 적용하고 1차원 구조로 재배열해 기존 셀프 어텐션 방법으로 학습하는 방식으로 진행된다. 이를 통해 CVT 모델은 지역 특징을 학습할 수 있고, 시퀀스 길이를 줄이는 동시에 토큰 특징의 차원을 증가시킨다. 합성곱 모델에서 수행되는 방식처럼 다운 샘플링과 특징 맵의 수를 늘리게 된다.

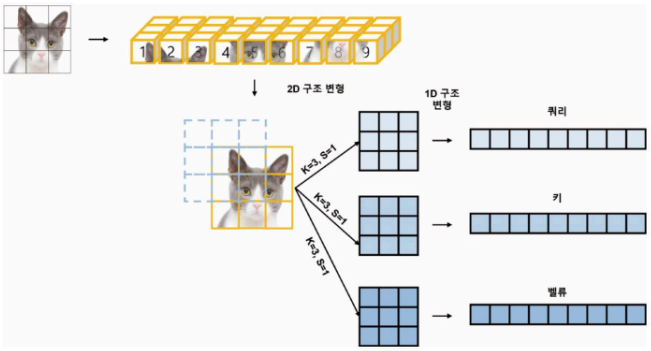


그림 10.15 2D 합성곱 기반의 어텐션 임베딩

- 어텐션에 대한 합성곱 임베딩 : 2D로 재구성된 토큰맵에서 분리 가능한 합성곱 연산을 적용해 성능 저하 및 계산 복잡도를 낮추는 연산으로, 매개변수의 수를 줄이기 위해 쿼리, 키, 값의 차원을 축소시킨 것이다.

  그림에서 쿼리는 3x3 커널 크기, 1 간격의 커널을 적용해 9개의 이미지 패치를 생성하며, 키와 값은 3x3 커널 크기, 2간격으로 커널을 적용해 4개의 이미지 패치를 생성한다.

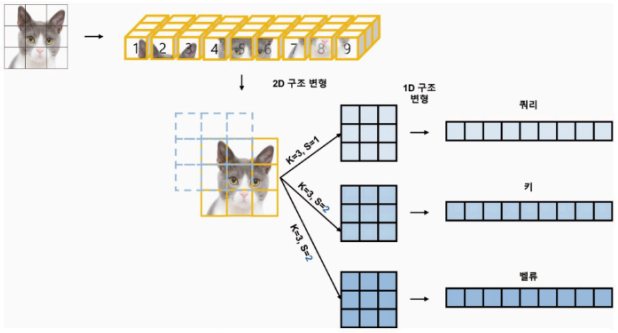


그림 10.16 2D 합성곱 기반의 축소된 어텐션 임베딩

길이가 다른 쿼리(9), 키(4), 값(4)을 사용하더라도 기존 셀프 어텐션의 결과값 사이즈가 9x1로 동일하게 출력될 수 있다.

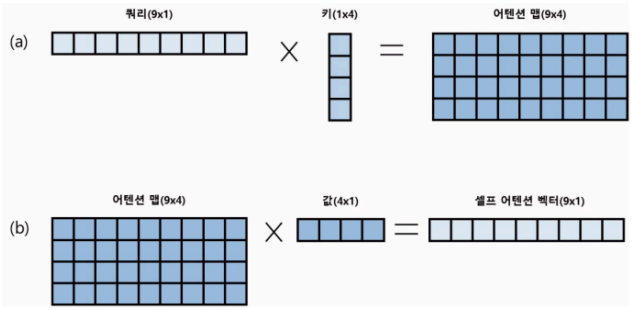


그림 10.17 축소된 셀프 어텐션 임베딩(열 × 행) 예시

그림 (a)에서는 9개의 쿼리 패치 임베딩과 4개의 키 패치 임베딩에 대한 중요도를 의미하는 어텐션 맵을 생성한다ㅏ. 어텐션 맵이 생성되면 (b)와 같이 값 패치 임베딩으로 쿼리 어텐션 맵에 대한 가중치를 반영한다. 이때 셀프 어텐션 벡터를 확인해보면 입력 벡터와 출력 벡터의 패치 길이가 동일한 것을 확인할 수 있다. 그러므로 계산해야 하는 이미지 패치 개수가 줄어들어 연산량이 감소한다.

CVT 모델은 트랜스포머 계층이 깊어질수록 패치 간의 관계를 학습하는 길이를 축소시키는 대신에 벡터의 차원을 증가시켜 모델의 매개변수 수를 대폭 감소시킨다. 예를 들어 첫 번째 트랜스포머 블록의 이미지 패치 크기가 64×64, 임베딩 차원이 128이고 마지막 트랜스포머 블록의 이미지 패치 크기를 4배 감소시켜 8×8로 설정했다면, 기존 임베딩 차원도 4배 증가시켜 512차원으로 증가시킨다. 다시 말해 이미지 패치 크기가 줄어든 만큼 임베딩 차원을 늘리는 방법을 택해 네트워크의 안정성을 향상시키고 모델 설계를 단순화 한다.